



MACHINE LEARNING

ASSIGNMENT 1

Bayesian Reasoning, Naive Bayes Classification & K-Nearest Neighbors

Total: 135 marks + 8 bonus (Question 1: 20 marks | Question 2: 115 marks + 8 bonus)

Submission Date: **Sunday October 11, 2026 at 11:59pm**

Assignment Rules:

1. AI Integrity

- You may consult documentation, lecture notes, and AI tools to learn concepts. However, every number, plot, and hand written test sentence in your submission must come from your own run of your own code, on your own data split.
- If your use of AI tools becomes apparent during the viva/demo, you must be able to explain and defend the code you submitted including how it works and why you implemented it that way. Inability to explain your own submitted code will be treated as a failed viva for the relevant part(s), *regardless of how the code was produced*.
- Plagiarism (from classmates, prior submissions, or code you cannot personally explain) results in **zero marks** for the affected part, per course policy.

2. Library Restrictions

- Allowed: **numpy**, **pandas**, **matplotlib** (or **seaborn**) for computation and plotting, and Python's standard library (**re**, **collections**, **string**, **math**).
- Not allowed for your core Naive Bayes and KNN implementations (Parts A–F): **scikit-learn**, NLTK's classifiers, or any pre-built Naive Bayes / KNN / text-classification function. You are implementing the counting, distance, and probability logic yourself.
- scikit-learn** is permitted in exactly one place Part G.1, as a labeled sanity-check baseline and nowhere else.
- Not allowed anywhere: pre-trained sentiment tools or language models (e.g., VADER, spaCy's sentiment, transformer-based classifiers). This assignment is about Naive Bayes and KNN, not about the best possible sentiment accuracy.
- For Question 1 (Notebook 1), the library and preprocessing rules stated in the notebook apply. The Bayesian classifier (categorical likelihood tables, Gaussian statistics and log-sum-exp normalization) must be implemented by you; pre-built Naive Bayes classifiers are not allowed.

3. Originality of Work

- Every student must be able to explain their own code if asked during viva/defense.

4. Submission Guidelines

- Submit as a single Jupyter Notebook (.ipynb) with all cells executed and outputs/plots visible.
- Late submissions will not be accepted unless officially approved.
- File naming convention should be **24i-XXXX_firstname_Section**.
- Question 1 is completed in the provided Notebook 1 (.ipynb) and Question 2 in its own notebook (.ipynb); every cell in each notebook must be executed with outputs visible.

5. Individual Assignment

- a. Keep in mind that this is an individual assignment and you can't form groups.

QUESTION 1 — Austin Animal Center: Bayesian Classification (Mixed Features) [20 marks]

Problem Statement

Animal shelters often face the challenge of predicting the outcomes of animals based on the information available at intake. Accurate outcome prediction is useful for understanding likely shelter outcomes, allocating resources, and supporting decision-making regarding animal care. For this assignment, you will use the Austin Animal Center Shelter Intakes and Outcomes dataset from Kaggle. The dataset contains historical shelter intake and outcome records with attributes describing the animal, intake circumstances, timing, and eventual outcome. Relevant attributes include:

- **Categorical attributes:** animal type, sex upon intake, breed, color, intake type, intake condition.
- **Numeric/derived attributes:** age in days at intake and time in shelter in days.
- **Target variable:** a simplified outcome class derived from the outcome type.

To do:

You are required to complete the notebook that will be provided to you:

Notebook 1 – Bayesian Classification (Mixed Features)

- a. Implement a Bayesian classifier with categorical + Gaussian features.
- b. Strict preprocessing rules must be followed.

Dataset

Austin Animal Center Shelter Intakes and Outcomes dataset (Kaggle):

<https://www.kaggle.com/datasets/aaronschlegel/austin-animal-center-shelter-intakes-and-outcomes>

Use this same Austin dataset for Notebook 1. The dataset contains approximately 80,000 shelter intake/outcome records.

Required Target Mapping

For Notebook 1, derive a simplified target variable from the dataset's Outcome Type field:

- Adoption → Adoption
- Transfer → Transfer
- Return to Owner → Return_to_Owner
- All remaining outcome categories → Other

Use the resulting four-class target consistently wherever the assignment asks for the classification target.

Required Derived Features

- **age_days:** convert the animal's age at intake into an equivalent number of days.
- **time_in_shelter_days:** calculate the difference between outcome date/time and intake date/time in days.
- Handle missing or invalid values according to the preprocessing instructions in the notebook. Do not silently invent values.

Evaluation

You will evaluate model performance using:

- Accuracy
- Precision, Recall, F1 (per class)
- Macro-F1
- Confusion Matrix

2. Learning Objectives

By the end of this assignment, you should be able to:

Question 1 — Austin Shelter Outcomes (Mixed Naive Bayes)

- Implement a Bayesian classifier from scratch that combines categorical features (with Laplace smoothing) and numeric features (Gaussian likelihood) in a single model.
- Apply strict, rule-based preprocessing target mapping, derived features (age_days, time_in_shelter_days), prescribed encodings, a stratified 80/20 split with a fixed seed, and z-scoring of numeric features without silently inventing values for missing or invalid data.
- Build categorical likelihood tables with Laplace smoothing using your personal K, and estimate Gaussian parameters (mean and variance) with a variance clamp to keep the model numerically stable.
- Compute class posteriors in log space and normalize them with a stable log-sum-exp.
- Evaluate a four-class, imbalanced classifier using accuracy, per-class precision/recall/F1, macro-F1 and a confusion matrix, and interpret these results.
- Make the model's behavior transparent by logging class counts, top-20 breeds, log-priors and a fully worked test-row breakdown, and summarize the work in a structured report.

3. Assignment Instructions

Question 1 — Notebook 1 (Austin Shelter Outcomes)

- Work inside the Notebook 1 file provided to you and keep its section order; complete all required code and markdown cells.
- Apply the preprocessing rules exactly as specified in the notebook: the four-class target mapping, age_days and time_in_shelter_days derivations, the prescribed encodings, the stratified 80/20 split with the given seed, and z-scoring of the numeric features. Do not silently invent or fill in missing/invalid values.
- Implement the classifier yourself: categorical likelihood tables with Laplace smoothing using your personal K, Gaussian likelihoods (mean/variance with variance clamp), and log-space posteriors normalized with a stable log-sum-exp. Pre-built Naive Bayes classes (e.g., from scikit-learn) must not be used for the model itself.
- Print the mandatory logs in the notebook output: class counts, top-20 breeds, log-priors, and one worked test-row breakdown (categorical log-likelihoods, Gaussian statistics and final posteriors).
- Report accuracy, per-class precision/recall/F1, macro-F1 and the confusion matrix from your own run.

Write the report using the fixed structure: Overview, Preprocessing, Model, Results, Analysis, Limitations, Worked Example.

QUESTION 2 — CineTrust Analytics: Auditing a Naive Bayes Review-Triage System [115 marks + 8 bonus]

1. Background & Case Study

CineTrust Analytics provides review-moderation tools to a mid-sized movie streaming platform. The platform's Trust & Safety team currently relies on a simple Naive Bayes classifier to automatically tag each incoming user review as positive or negative sentiment. The tagged output feeds a dashboard that flags unusual spikes in negative sentiment around a title's release, which analysts use to catch bugs, botched releases, or coordinated review-bombing early.

The classifier has never been formally reviewed. Before it is scaled up and given more influence over moderation decisions, the engineering lead has asked your team to audit it not just to report an accuracy number, but to investigate whether its core assumptions actually hold on real review text, whether its confidence scores can be trusted by downstream analysts, and how it behaves when the real-world mix of positive and negative reviews differs from the balanced dataset it was trained on.

You will build the classifier yourself (in two variants), stress-test it with deliberately hard examples, benchmark it against a fundamentally different, non-parametric approach (K-Nearest Neighbors), and end by writing a short, evidence-based recommendation to the engineering lead: is this model ready to ship, and if not, what specifically needs to change?

2. Learning Objectives

Question 2 — CineTrust Review Audit (IMDB)

- Derive and correctly apply Bayes' theorem to a text classification problem, including stating the conditional-independence assumption explicitly.
- Implement Multinomial and Bernoulli Naive Bayes classifiers from scratch using bag-of-words features, and explain when each representation is more appropriate.
- Apply Laplace / add-k smoothing correctly and explain, with evidence, why it is required.
- Investigate how the (false) independence assumption affects not just accuracy but the reliability of predicted probabilities (calibration).
- Adjust a trained Naive Bayes model for a deployment-time class prior that differs from the training prior, without retraining.
- Diagnose specific, concrete failure modes of bag-of-words models (negation, sarcasm, correlated features) through hand-designed test cases.
- Implement a non-parametric K-Nearest Neighbors classifier from scratch and critically compare its accuracy, calibration behavior, and computational cost against parametric Naive Bayes models.
- Communicate a quantitative ML finding as a short, evidence-backed recommendation for a non-technical stakeholder.

3. Assignment Instructions

Question 2 — CineTrust (IMDB)

Read this entire document before writing any code. Work through the parts in order later parts assume you have completed the classifiers from Part B, and Part F assumes the feature vectors you built for Part B.

The rules governing academic integrity, allowed libraries, originality, and submission format are given above under "Assignment Rules." Read them before starting.

4. Dataset Requirements

You will use the IMDB movie review polarity dataset.

- 2,000 plain-text reviews total: 1,000 positive and 1,000 negative (one review per .txt file).
- Label (target variable): binary sentiment, derived from which folder a file is in (pos = 1, neg = 0). There is no separate label column you must construct labels() from folder membership.
- Features: you will build a bag-of-words vocabulary yourself from the training portion of the data only. There are no pre extracted numeric features to load feature engineering is part of the assignment. The same feature vectors are reused for the KNN classifier in Part F.
- Review length varies considerably (from very short to over 2,500 words); do not assume all reviews are a similar length this variation is directly relevant to the Multinomial vs. Bernoulli comparison in Part B, and to the choice of distance metric in Part F.
- You must create your own stratified train / validation / test split (a common choice is 70% / 15% / 15%), fix and report a random seed, and keep the vocabulary construction confined to the training split only to avoid leaking test-set words into your model.

5. Choice of Naive Bayes Variant

Gaussian Naive Bayes is not appropriate here review text does not naturally produce continuous, normally distributed features. Text is naturally represented as either word counts or word presence/absence, which corresponds to two different, both-valid modeling choices:

- Multinomial Naive Bayes — models how many times each word occurs; a natural fit given these reviews average around 750 words, so frequency carries information (e.g., a review that says “great” five times is a different signal than one that says it once).
- Bernoulli Naive Bayes — models only whether each vocabulary word is present or absent, ignoring repetition; simpler, and sometimes more robust when review length varies a lot, but throws away frequency information.

Rather than mandating one, this assignment requires you to implement both and let your own results in Part B decide which performs better on this dataset with your explanation grounded in the data (Part B.6), not in a textbook default. Part F then asks whether a completely different kind of model one that makes no independence assumption at all changes that conclusion.

6. Questions / Tasks

Part A — Foundations: Bayes’ Theorem & Log-Space Reasoning (15 marks)

1. Starting from Bayes’ theorem, derive the Naive Bayes decision rule for binary text classification. State explicitly which conditional independence assumption is being made, and exactly where in the derivation it is used.
2. By hand (show every step no code for this sub question), use the following 4 document toy corpus with add-1 (Laplace) smoothing to compute $P(\text{pos} \mid \text{“fun great”})$ and $P(\text{neg} \mid \text{“fun great”})$, and state the predicted class.
 - D1 (pos): “fun fun great”
 - D2 (pos): “great acting fun”
 - D3 (neg): “boring boring bad”
 - D4 (neg): “bad acting boring”
 - Test review: “fun great” | Vocabulary: {fun, great, acting, boring, bad}

You will re run this exact toy example through your own code in Part B.8 your two answers must match, so keep your work.

3. Explain, in your own words and using a number from the real IMDB dataset (e.g., roughly how many word likelihood terms get multiplied together for an average length review), why Naive Bayes implementations compute in log probability space rather than multiplying raw probabilities directly. What specifically goes wrong numerically if you don't?

Part B — Implementation: Multinomial & Bernoulli Naive Bayes From Scratch (25 marks)

4. Implement text preprocessing: tokenization and cleaning. Decide whether to remove stopwords, punctuation only tokens, or numbers, and justify each choice a choice without a justification earns no credit here.
5. Build your vocabulary from the training split only. Report vocabulary size before and after any filtering (e.g., a minimum document-frequency threshold), with the actual counts.
6. Implement a MultinomialNB class from scratch with fit, predict, and predict_log_proba methods, computed in log space.
7. Implement a BernoulliNB class from scratch the same way, using word presence/absence features. (Remember: Bernoulli NB's likelihood must also account for vocabulary words that are absent from a document, not only the words present this is the most common implementation bug.)
8. Cross-check: run your MultinomialNB on the Part A.2 toy corpus and test review. Report your code's $P(\text{pos} \mid \text{test})$ and $P(\text{neg} \mid \text{test})$ side-by-side with your hand-calculated values from A.2, and state whether they match (they should, within floating-point rounding).
9. Train and evaluate both classifiers on your IMDB split. Report accuracy, precision, recall, F1-score, and a confusion matrix for each. State which variant performs better and by how much, and propose one data grounded reason why (e.g., relate it to the review length distribution or to vocabulary sparsity a guess with no reference to your own data earns partial credit only).

Part C — Smoothing, Vocabulary, and Priors as Design Decisions (18 marks)

10. Add-k smoothing sweep: retrain MultinomialNB with several values of k (e.g., 0.01, 0.1, 0.5, 1, 2, 5). Plot accuracy vs. k . Using your own plotted numbers, explain the bias-variance trade-off visible at each extreme (k too small vs. k too large).
11. Out-of-vocabulary stress test: assemble at least 10 test reviews (from your held out data or hand written) containing words absent from your training vocabulary. Show, with actual numbers, what happens to your classifier's output with smoothing turned off vs. on. Explain in probabilistic terms why the unsmoothed version fails the way it does.
12. Prior-probability sensitivity deployment scenario: your engineering lead reports that in real traffic, historically 68% of submitted reviews are positive (unlike your balanced 50/50 training data).
 - (a) Without retraining your likelihood parameters, recompute predictions on your test set using a 0.68 / 0.32 prior instead of the training-derived prior.
 - (b) Report how many test predictions flip as a result, and show at least 3 concrete flipped examples with their log-posterior scores before and after.
 - (c) Discuss: is adjusting the prior always the right move for a platform like this? What could go wrong if the "68% positive" figure itself were inaccurate, seasonal, or manipulated (e.g., by coordinated fake reviews)?

Part D — Are the Probabilities Trustworthy? Calibration & Overconfidence (14 marks)

13. For every test review, record the model's predicted probability (confidence) for its predicted class. Group reviews into confidence bins (e.g., 0.5–0.6, 0.6–0.7, ..., 0.9–1.0) and compute the actual accuracy within each bin. Plot a reliability diagram (predicted confidence vs. observed accuracy).
14. Naive Bayes is known for producing overconfident probabilities (pushed toward 0 or 1) even when accuracy is only moderate. Does your reliability diagram show this? Explain, mechanically, why multiplying many falsely independent likelihood terms produces this effect and back your explanation with your own bin counts, not a general statement.
15. Propose (describe implementation is not required) one practical mitigation CineTrust could apply before using these scores for confidence-based flagging (e.g., Platt / temperature scaling, or restricting automated action to a narrow high-confidence band). Justify it directly against what you observed in D.1–D.2.

Part E — Structural Failure Modes: Negation, Sarcasm, and Correlated Words (13 marks + 8 bonus)

16. Write, yourself (do not copy from the internet), at least 8 short test sentences designed to break the bag-of-words / independence assumption: at least 2 negation cases (e.g., “not good”, “wasn't terrible”), 2 sarcasm/irony cases, and 2 cases built from words that are individually mild but jointly change the meaning when combined. Run both of your classifiers on all 8+ sentences and report the predicted class and confidence for each.
17. For every sentence your model gets wrong, explain precisely which modeling assumption caused the failure, and why bag-of-words cannot fix this without a feature-engineering change. Propose one concrete feature change (e.g., bigrams, or negation-tagging such as turning “not good” into a single token “not_good”) that could plausibly help and justify it with reference to your specific failing sentences.
18. [BONUS up to 8 marks] Implement your proposed fix from E.2(17) for at least one failure mode, and report honestly whether it improves, worsens, or does not change your Part E.1(16) predictions. A wellreasoned negative result earns full bonus credit; an unsupported claim of improvement without a shown before/after comparison earns none.

Part F — Extension: K-Nearest Neighbors as a Non-Parametric Baseline (15 marks)

Naive Bayes assumes conditional independence between words and a fixed parametric form for each class conditional likelihood. K-Nearest Neighbors makes neither assumption it classifies a review by looking at the labels of the most similar reviews in the training set. This part asks whether that fundamentally different approach changes the audit's conclusions.

19. Implement a KNNClassifier class from scratch (fit, predict, and predict_proba methods) operating on the bag-of-words feature vectors you built in Part B. Use cosine distance as your primary metric, and justify why cosine distance is generally preferred over raw Euclidean distance for high-dimensional, variable length text vectors relate your justification to the review-length variation you documented in Section 4.
20. Select k using your validation split only (never the test split): sweep at least 5 candidate values of k (e.g., 1, 3, 5, 15, 31), plot validation accuracy vs. k, and choose your final k with a justification grounded in the bias-variance trade-off visible in your plot.
21. Train your tuned KNN classifier on the same train/test split used for your Naive Bayes classifiers in Part B. Report accuracy, precision, recall, F1-score, and a confusion matrix, and present these results in a single table alongside your MultinomialNB and BernoulliNB numbers from Part B.9. State, with reference to your own numbers, which of the three models performs best and by how much.
22. Naive Bayes predicts in roughly linear time per document (summing precomputed log-likelihoods), while a naive KNN implementation must compare a query review against every training document. Measure

and report the average prediction time per test review for your KNN implementation versus your MultinomialNB implementation, and discuss what this measurement means for CineTrust if the review moderation system needed to score reviews in real time as they are submitted.

Part G — Sanity-Check Baseline & Final Recommendation (15 marks)

23. As a sanity check only, run scikit-learn's MultinomialNB, BernoulliNB, and KNeighborsClassifier with default settings on the exact same split, and report their metrics next to your from-scratch results (Parts B and F) in one table. Your numbers should be broadly comparable (not necessarily identical, given default-parameter differences) briefly explain any notable gap.
24. Write a roughly 1–1.5 page recommendation, addressed to “the CineTrust engineering lead”, that: states whether you would ship this classifier now, ship it with specific conditions, or not ship it at all; cites at least 3 specific numeric findings from Parts C–F as evidence (not vague impressions); and names the single biggest risk you found, with one concrete next step to address it.

7. Technical Requirements

- Python 3, submitted as a single Jupyter Notebook (.ipynb) with all cells executed and outputs/plots visible.
- Libraries: numpy, pandas, matplotlib/seaborn, and Python's standard library — required for Parts A–F. scikit-learn is allowed only in Part G.1, clearly labeled.
- No pre-trained sentiment or NLP models anywhere in the assignment (see Assignment Rules, item 2).
- Fix and report a random seed wherever randomness is used (train/test split, any sampling, k-fold validation for Part F).

8. Expected Deliverables

Question 1 — Notebook 1 (Austin Shelter Outcomes)

- The completed Notebook 1 (.ipynb) with all cells executed and all outputs visible.
- The mandatory logs printed in the output: class counts, top-20 breeds, log-priors, and the worked test-row breakdown (categorical log-likelihoods, Gaussian statistics and final posteriors).
- The evaluation results: confusion matrix, accuracy, per-class precision/recall/F1 and macro-F1.
- The report following the fixed structure (Overview, Preprocessing, Model, Results, Analysis, Limitations, Worked Example), either as a final markdown section in the notebook or as a separately attached document, per instructor preference.

Question 2 — CineTrust (IMDB)

- One Jupyter Notebook (.ipynb) containing all code, executed outputs, and all plots (rendered inline, not just generated locally), organized under clearly labeled sections matching Parts A–G.
- Markdown explanations for every task, not just code without commentary does not satisfy the written answer requirements above.
- The Part G.2 recommendation report, either as a final markdown section in the notebook or as a separately attached document, per instructor preference.

Question 1 — Notebook 1: Bayesian Classification (Mixed NB, Austin Shelter Outcomes) [20 marks]

Category	Criterion	Marks
Data Preprocessing & Rules Compliance (5)	Correct derivation of age_days, time_in_shelter_days	1
	Strict encodings applied exactly as specified	2
	Stratified 80/20 split with correct seed	1
	Correct z-scoring of numeric features	1
Model Implementation (7)	Laplace smoothing with personal K applied correctly	2
	Categorical NB likelihood tables correct	2
	Gaussian NB (mean/variance with variance clamp) correct	2
	Stable log-sum-exp for normalization	1
Evaluation (4)	Confusion matrix correct	1
	Accuracy correct	1
	Per-class Precision/Recall/F1 correct	1
	Macro-F1 correct	1
Mandatory Logs (2)	Printed class counts, top-20 breeds, log-priors	1
	Printed worked test row breakdown (categorical log-likes + Gaussian stats + final posteriors)	1
Report (2)	Fixed structure followed (Overview, Preprocessing, Model, Results, Analysis, Limitations, Worked Example)	2
Total		20

- Marks within each part require both a correct numeric/experimental result AND a written explanation grounded in that result a correct number with no interpretation, or an interpretation not backed by a shown number, earns partial credit only.
- Code that runs but produces no markdown commentary explaining what was done and why will lose marks under each part, independent of correctness.

9. Evaluation Criteria

Part	Focus	Marks	Bonus
Q1	Question 1 (Notebook 1) — Bayesian Classification (Mixed NB): preprocessing, model implementation, evaluation, mandatory logs, report	20	—
A	Foundations — Bayes' theorem derivation, hand worked toy example, log-space reasoning	15	—
B	Implementation — Multinomial & Bernoulli Naive Bayes from scratch, evaluation	25	—
C	Smoothing sweep, OOV stress test, deployment-prior sensitivity	18	—
D	Calibration / reliability diagram, overconfidence analysis	14	—
E	Hand-crafted adversarial tests, failure-mode analysis	13	8
F	K-Nearest Neighbors baseline — from-scratch implementation, k-tuning, comparison to Naive Bayes, runtime analysis	15	—
G	Baseline sanity check and written recommendation report	15	—
Total		135	8